DATA ENGINEERING REPORT · PROJECT 0

# Ubisoft vs Electronic Arts

What Steam data reveals:
an end-to-end ETL pipeline, from client brief to acceptance

**LUMU BAMANA MANASSE**

Data engineer

| | |
|---|---|
| **Date** | October 2026 |
| **Version** | 1.0.0 (delivered and accepted) |
| **Client** | Lumen Forge Games (fictional) · Engagement LFG-2026-001 |
| **GitHub repository** | https://github.com/MediJr/ubisoft-vs-ea-steam-etl |
| **Language** | English · *version française disponible dans le dépôt* |

Fictional scenario, real data: the client and its staff are invented to simulate a complete professional engagement. The data comes from the public Steam and SteamSpy APIs, extracted on 21 October 2026. This project is not affiliated with Ubisoft, Electronic Arts or Valve.

## Document control

| | |
|---|---|
| **Project** | Portfolio project 0: ETL pipeline on public Steam data |
| **Engagement** | LFG-2026-001: "Understand what works for major publishers on Steam" |
| **Client** | Lumen Forge Games, independent studio based in Lyon, France (fictional), attn. Camille Roux |
| **Author** | Lumu Bamana Manasse, data engineer |
| **Delivery period** | October 2026 |
| **Data** | Extraction of 21 October 2026, 18:38 to 19:10 (Paris time) |
| **Status** | DELIVERED ACCEPTED WITH MINOR RESERVATIONS |
| **Source code** | https://github.com/MediJr/ubisoft-vs-ea-steam-etl |

### Version history

| Version | Date | Description |
|---|---|---|
| 0.1 | 2026-10-21 | Scoping: brief, meeting minutes, statement of work |
| 0.9 | 2026-10-21 | Complete pipeline, analyses, data quality fixes |
| 1.0.0 | 2026-10-22 | Acceptance testing, final report (French and English), publication |

### Reading guide

| | |
|---|---|
| **Decision maker (client)** | 1. Executive summary and 11. Results and recommendations |
| **Data lead** | 2. Scoping, 4. Architecture, 7. Data quality, 10. Acceptance, 12. Limitations |
| **Engineer taking over** | 5. Pipeline, 6. Data model, 9. Operations, appendices |

## Contents

**Document control** 2

**1 Executive summary** 4

**2 Context and scoping** 4
- 2.1 From a vague request to measurable questions . . . 4
- 2.2 Scope . . . 5
- 2.3 Approach . . . 5

**3 Data sources** 6

**4 Solution architecture** **6**
4.1 Technology choices . . . 7
4.2 Design principles . . . 7

**5 Pipeline design** **7**
5.1 Extraction . . . 7
5.2 Transformation . . . 8
5.3 Loading . . . 8

**6 Data model** **9**

**7 Data quality** **9**
7.1 Built-in checks . . . 9
7.2 Issues detected and fixed . . . 10

**8 Security, compliance and governance** **10**

**9 Operations** **11**
9.1 Commands . . . 11
9.2 Capture of a full run . . . 11
9.3 Incident and recovery . . . 11

**10 Testing and acceptance** **12**

**11 Results and recommendations** **13**
11.1 Q1 · Who best satisfies its players, and for how long? . . . 13
11.2 Q2 · Are more expensive games rated higher? . . . 14
11.3 Q3 · Which games keep their players? . . . 15
11.4 Q4 · Do French-speaking players judge differently? . . . 16
11.5 Recommendations to the client . . . 16

**12 Limitations and technical debt** **17**

**13 Lessons learned** **17**

**14 Roadmap** **18**

**A Data dictionary: `games` table** **18**

**B Architecture decision records (ADR)** **18**

**C Repository structure** **19**

**D Glossary** **19**

## 1 Executive summary

**The need.** Lumen Forge Games, an independent studio with no data team, is preparing to release its first game on Steam at around €25. Before setting its price and launch strategy, it wants to learn what works (and what fails) for two major publishers: **Ubisoft** and **Electronic Arts**.

**The solution.** A Python ETL pipeline automatically collects public data from Steam and SteamSpy (six sources), cleans it and loads it into a DuckDB analytical database. Ten SQL queries answer the four business questions, and a report presents the findings. Everything runs with **a single command**.

| | | | |
|---|---|---|---|
| **100** games analysed (50 per publisher) | **9.8 M** Steam reviews aggregated | **930** raw API responses kept | **0** personal data collected |
| **6** data sources | **30 min** full extraction (fresh clone) | **2 s** batch replay with no API calls | **6 / 6** acceptance criteria met |

### Key findings

- **Q1 · Satisfaction:** a tie on average (80%), but two opposite profiles. EA is "hit or miss" (85% median, yet 11 games below 70%). Ubisoft is more consistent, and its games **gain 3.2 points after release**.
- **Q2 · Price:** the more expensive a game, the lower its rating (correlation of about $-0.5$). Above €60, satisfaction drops.
- **Q3 · Longevity:** the games that last are free-to-play or multiplayer.
- **Q4 · French speakers:** slightly more generous (+2 points), but they write only 2% of reviews.

**What this project demonstrates on the engineering side:** API-friendly extraction (throttling, retries, resume cache), an immutable dated raw layer, transactional loading with no duplicates, privacy *by design* (no player username is ever downloaded), detection and correction of four data quality issues in real-world data, and a documented incident recovery.

## 2 Context and scoping

### 2.1 From a vague request to measurable questions

The initial request fit in one sentence: "understand what works for the big publishers". The scoping meeting turned it into four questions, each tied to a concrete client decision.

| # | Business question | Decision supported |
|---|---|---|
| Q1 | Which of the two publishers best satisfies its players on Steam, and is that satisfaction stable over time? | Post-launch reputation monitoring |
| Q2 | Are more expensive games rated higher? Are discounts a sign of trouble? | Launch price and discount policy |
| Q3 | Which games keep their players years after release? | Post-launch content |
| Q4 | Do French-speaking players judge these publishers differently? | Priority market |

**Table 1:** Business questions agreed during scoping

## 2.2 Scope

| In scope | Out of scope |
|---|---|
| Games of type `game` published on Steam by Ubisoft or EA | DLC, soundtracks, demos, bundles |
| At least 500 reviews, at most 50 games per publisher (most reviewed) | Other platforms: EA app, Ubisoft Connect, consoles |
| Store pages, review summaries, monthly history, player counts | Review text and any personal data |
| Analysis report and re-runnable pipeline | Stock market impact, interactive dashboard, automated scheduling |

## 2.3 Approach

The project follows the five phases of a data consulting engagement. Each phase produces deliverables versioned in the repository.

| Phase | Purpose | Deliverables |
|---|---|---|
| 1 | Scoping | Client brief, scoping minutes, statement of work (`docs/mission/`) |
| 2 | Extraction | `src/extract.py`, raw layer `data/raw/`, ADR 002 |
| 3 | Transformation and loading | `src/transform.py`, `src/load.py`, `sql/00_schema.sql` |
| 4 | Analysis | 10 SQL queries, 5 charts, auto-generated results |
| 5 | Delivery | Acceptance (sign-off), this report, documentation, publication |

**Table 2:** Engagement phases

## 3 Data sources

| Source | Status | Data used | Purpose |
|---|---|---|---|
| Steam Store search | Unofficial | List of a publisher's games (`app_id`) | Scope |
| Store page (`appdetails`) | Public, undocumented | Publisher, price in euros, discount, genres, release date | Q2, Q3 |
| Reviews (`appreviews`) | Public, documented | Review totals, all languages and French | Q1, Q4 |
| History (`appreviewhistogram`) | Unofficial | Positive and negative reviews per month | Q1 |
| Steam Web API | Official | Concurrent players | Q3 |
| SteamSpy | Third party | Owner estimates, previous-day peak | Information |

**Table 3:** Source inventory

**Constraints.** Steam tolerates about 200 requests per 5 minutes. Three endpoints are unofficial and may change without notice. No API key is required, so there are no secrets to manage.

### Upfront exploration: test before committing

Before signing the statement of work, every endpoint was tested by hand. The `day_range` parameter of the reviews API *seems* to filter reviews over a period, but in fact it **does not change** the returned summary. Without that test, the stability measure (Q1) would have been promised on a false premise. The monthly history (`appreviewhistogram`) is what made it possible.

## 4 Solution architecture

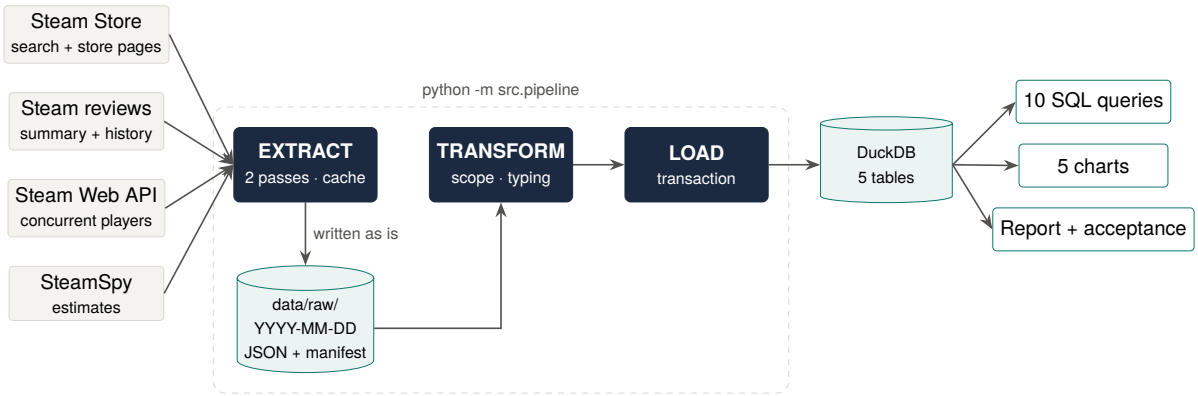



**Figure 1:** Pipeline architecture: one command chains extraction, transformation, loading and report generation

### 4.1 Technology choices

| Need | Choice | Rationale and rejected alternatives |
|---|---|---|
| Language | Python 3.14 | Data engineering standard, rich ecosystem |
| HTTP calls | `requests` | Simple and reliable. `httpx` (async) is unnecessary at this volume. |
| Transformation | `pandas` | Fits a few thousand rows. Polars is faster but brings no gain here. |
| Storage | DuckDB (file) | Serverless analytical SQL. PostgreSQL will be the novelty of project 1. |
| Analysis | SQL | Readable, verifiable, independent from Python |
| Charts | `matplotlib` | Static figures embedded in the report |
| Environment | `venv` + `pip` | Universal. `uv` and Poetry come later. |
| Orchestration | None (one command) | Deliberate at level 0. Dagster is planned for project 3. |

**Table 4:** Technology stack (details: ADR 001)

### 4.2 Design principles

1. **Immutable raw layer:** API responses are stored unchanged. A transformation bug is fixed by replaying the batch, without calling the APIs again.
2. **Idempotency:** running the pipeline twice gives the same result, with no duplicates (primary keys, full refresh).
3. **Privacy by design:** personal data is never requested from the APIs.
4. **Reproducibility:** every figure in this report comes from a versioned query. `resultats.md` is regenerated on every run.
5. **Traceability:** one manifest per batch, an exclusions table with reasons, and three ADRs for the key decisions.

## 5 Pipeline design

### 5.1 Extraction

Collecting all six sources for the 160 candidate games would have taken about 1,000 calls, more than 25 minutes, for games that would partly be discarded anyway. Extraction therefore runs in **two passes**:

| Step | Action | Ubisoft | EA |
|---|---|---|---|
| Discovery | Steam search by publisher, “games” category | 80 | 80 |
| Light pass | Review summary (1 call per game), “at least 500 reviews” filter | 77 | 76 |
| Detailed pass | 5 calls per selected game | 77 | 76 |

**Table 5:** Extraction funnel (batch of 2026-10-21)

**Robustness.** Each call is preceded by a 1.5 s pause. HTTP 429 (too many requests) and 5xx errors trigger up to three retries with increasing back-off. An explicit `User-Agent` identifies the project. Every response is cached in the raw layer, so an interrupted extraction resumes where it stopped. Writes are **atomic**, a change made after the incident described in section 9.3:

```
temporary = path.with_suffix(".tmp")
temporary.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
temporary.replace(path)   # instant rename: never a half-written file
```

**Privacy at the source.** Reviews are requested with `num_per_page=0`: Steam returns totals only, with no individual review, hence no username or identifier.

## 5.2 Transformation

| Rule | Implementation |
|---|---|
| Scope | Type `game`, already released, actual publisher matches the searched publisher |
| Cap | The 50 most reviewed games per publisher. The others are logged in `exclusions` (53 games). |
| Typing | French dates (“12 janv. 2023”) parsed to `DATE`, prices in euros, SteamSpy ranges to integers |
| Missing values | Review rate `NULL` rather than 0 when there are no reviews. A free game costs €0. |
| History | Unix timestamps converted to months, aggregated if Steam returns weekly buckets |

**Table 6:** Transformation rules

## 5.3 Loading

Loading uses a **full refresh** strategy: the schema is recreated, then every table is inserted, **within a single transaction**. On failure nothing is committed and the database keeps its previous state. It is never half-loaded.

```
connection.execute("BEGIN TRANSACTION")
try:
    connection.execute(SCHEMA_PATH.read_text(encoding="utf-8"))
    for name in LOAD_ORDER:                    # games first (foreign keys)
        connection.register("incoming", tables[name])
        connection.execute(f"INSERT INTO {name} BY NAME SELECT * FROM incoming")
```

```
        connection.unregister("incoming")
    connection.execute("COMMIT")
except Exception:
    connection.execute("ROLLBACK")
    raise
```

## 6 Data model




**Figure 2:** DuckDB data model (arrows represent foreign keys)

| Table | Grain (one row per...) | Rows |
|---|---|---|
| `games` | selected game | 100 |
| `review_summary` | game and language (`all`, `french`) | 200 |
| `reviews_monthly` | game and month | 9,929 |
| `player_snapshot` | game and extraction date | 100 |
| `exclusions` | discarded candidate, with its reason | 53 |

**Table 7:** Tables, grain and volumes (full data dictionary in appendix A)

## 7 Data quality

### 7.1 Built-in checks

- **Schema constraints:** primary keys, `NOT NULL`, foreign keys. An anomaly makes loading fail instead of skewing results.
- **Control query** (`sql/controle_recette.sql`): no duplicates, volume per publisher, review threshold, no personal column.
- **Exclusions table:** every discarded candidate has a reason. Nothing disappears without a trace.

### 7.2 Issues detected and fixed

Real-world data revealed four issues that reading the API documentation alone could not have anticipated.

| Symptom | Cause | Fix |
|---|---|---|
| 5 release dates not parsed | Steam writes "aout" (August) without its accent, and one page has no date | Month dictionary accepts both spellings. The missing date stays `NULL`. |
| 0 exclusions on first run: a result too perfect | Games dropped by the 50-game cap were not logged | 53 exclusions recorded with reasons |
| Player peak of 0 for 9 recent games (*Battlefield 6*: 0, vs. 45,100 online) | SteamSpy does not track them yet: 0 means "unknown" | Switched to the official Steam API (ADR 003) |
| 21 EA games "launched" in 2020 | EA returned to Steam in 2020, and some games predate Steam reviews | Q1b requires first reviews to match the Steam release date |

**Table 8:** Data quality issues found in real-world data

> **Lesson**
>
> A zero is not always a zero, and a perfect result is suspicious. Data quality cannot be assumed: it must be verified, dataset by dataset, before any analysis.

## 8 Security, compliance and governance

| Topic | Measure |
|---|---|
| Personal data (GDPR) | The reviews API can return usernames, identifiers and texts. Applying **data minimisation** (Art. 5(1)(c)) and **data protection by design** (Art. 25), only totals are requested. Verified: no `steamid` or `personaname` field in the 930 raw files, and no personal column in the database. |
| Secrets | No API key required. The `.gitignore` still excludes any `.env` file. |
| Published data | Data is not versioned (`data/` is ignored): it is regenerated by the pipeline. |
| API usage | Deliberately throttled, explicit `User-Agent`, non-commercial use |
| Traceability | One manifest per batch, dated work log, ADRs, CHANGELOG |

# 9 Operations

## 9.1 Commands

```
python -m venv .venv && .venv\Scripts\activate      # isolated environment (Windows)
pip install -r requirements.txt
python -m src.pipeline                              # today's extraction + processing (~30 min)
python -m src.pipeline --run-date 2026-10-21        # replay a raw batch, no API calls (~2 s)
```

## 9.2 Capture of a full run

Actual log of the acceptance run on a fresh environment (timestamps and paths shortened, intermediate progress hidden; the pipeline logs in French):

PowerShell · python -m src.pipeline

```
19:16:24 | INFO | src.extract   | Extraction vers ...\00-jeux-video-etl\data\raw\2026-10-21
19:16:29 | INFO | src.extract   | Ubisoft : 80 jeux candidats
19:19:17 | INFO | src.extract   | Ubisoft : 76 jeux avec au moins 500 avis
19:31:55 | INFO | src.extract   | Electronic Arts : 80 jeux candidats
19:34:39 | INFO | src.extract   | Electronic Arts : 76 jeux avec au moins 500 avis
19:46:47 | INFO | src.extract   | Extraction terminée
19:46:47 | INFO | src.transform | Table games            :    100 lignes
19:46:47 | INFO | src.transform | Table review_summary   :    200 lignes
19:46:47 | INFO | src.transform | Table reviews_monthly  :   9768 lignes
19:46:47 | INFO | src.transform | Table player_snapshot  :    100 lignes
19:46:47 | INFO | src.transform | Table exclusions       :     52 lignes
19:46:47 | INFO | src.load      | Base chargée : ...\data\steam.duckdb
19:46:49 | INFO | src.report    | Graphiques et résultats écrits dans ...\docs\livraison
19:46:49 | INFO | pipeline      | Pipeline terminé en 1825 s
```

## 9.3 Incident and recovery

| Time | Event |
|---|---|
| 18:38 | Production extraction started |
| 19:05 | **External interruption** of the process, at 56 of 76 EA games |
| 19:06 | Diagnosis: no running process, no manifest, 133 games cached. Check: no truncated JSON file. |
| 19:06 | Same command relaunched: the 133 cached games are re-read in **0.1 s**, then collection resumes |
| 19:10 | Extraction complete. Improvement: atomic writes, so that a hard stop can never leave a corrupted file. |

**Table 9:** Timeline of the 2026-10-21 incident

## 10 Testing and acceptance

Acceptance testing was run on a **fresh environment**: a copy of the publishable files only (38 files, no data, no virtual environment), a new Python environment, then the single command from the README.

| # | Criterion | Result | |
|---|---|---|---|
| C1 | End-to-end run with one command | Install in 54 s, pipeline in 30 min 25 s, exit code 0 | ✓ |
| C2 | No duplicates after a re-run | Same volumes before and after: 100 / 200 / 9,929 / 100 / 53 | ✓ |
| C3 | At least 15 games per publisher | 50 games per publisher, all with at least 500 reviews (minimum: 7,215) | ✓ |
| C4 | One query and one answer per question | 10 queries, one section per question | ✓ |
| C5 | No personal data | No personal field in 930 raw files or in the database | ✓ |
| C6 | Limitations stated | Section 12 | ✓ |

**Table 10:** Acceptance results (details: `docs/livraison/2026-10-21-pv-recette.md`, in French)

### Minor reservations

**R1 · Panel reproducibility.** Steam's search relevance order varies slightly between runs. Between production and acceptance, 99 of 100 games are identical. Key indicators match to one decimal place (weighted rates: EA 77.4%, Ubisoft 81.3%).

**R2 · Discounts.** No game was on sale on extraction day, so this part of Q2 cannot be answered.

# 11 Results and recommendations

## 11.1 Q1 · Who best satisfies its players, and for how long?

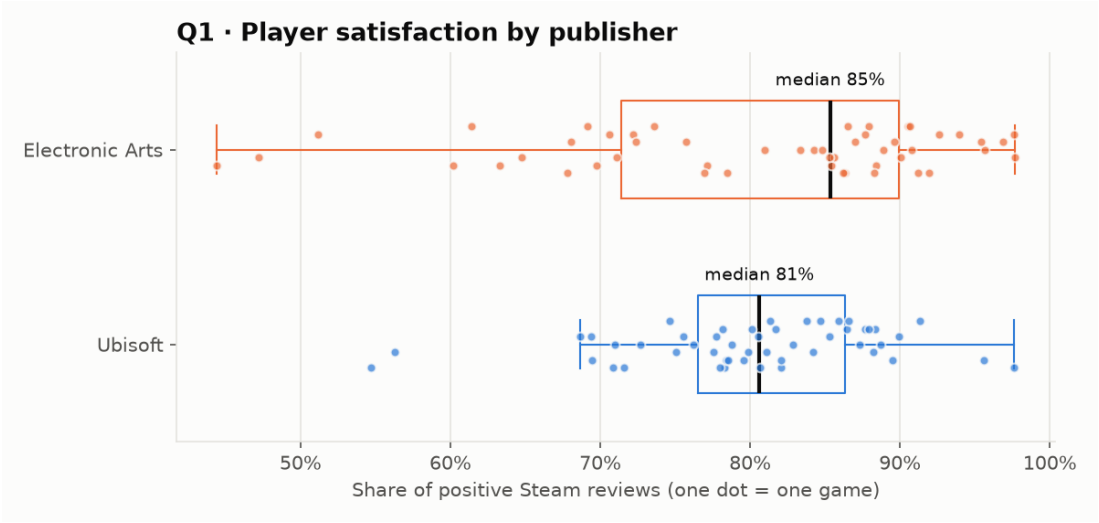



**Figure 3:** Share of positive reviews per game and per publisher

| Publisher | Mean | Median | Weighted | < 70% | Change | Recovered |
|---|---|---|---|---|---|---|
| **Electronic Arts** | 80.3% | 85.3% | 77.4% | 11 | −1.0 pt | 4 / 34 |
| **Ubisoft** | 80.2% | 80.6% | 81.3% | 5 | +3.2 pts | 12 / 39 |

**Table 11:** Overall satisfaction, then change between the first 3 months and the last 12 months on Steam

A tie on average, but two profiles: EA is more spread out (higher median, but twice as many games below 70%), whereas Ubisoft is more consistent and **improves its games after release** (*Assassin's Creed Unity* +24.5 points, *The Crew 2* +25.5).

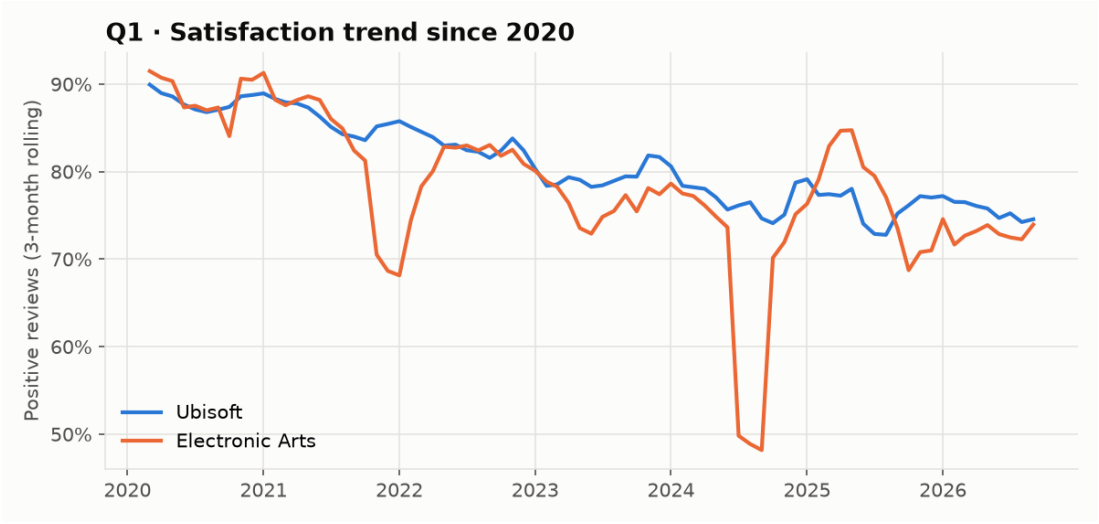



**Figure 4:** Monthly trend: two visible crises at EA

The monthly history reveals two crises at EA: the launch of *Battlefield 2042* (November 2021: 40,065 negative reviews, 35% positive, since recovered by +30.7 points), and a *review bombing* of *Apex Legends* in July 2024 (82,120 negative reviews in one month, 7% positive), six years after its release.

### 11.2 Q2 · Are more expensive games rated higher?




**Figure 5:** Current list price and satisfaction

No, rather the opposite: the correlation is $-0.55$ for EA and $-0.46$ for Ubisoft (Spearman: $-0.49$ for both). At €60 and above, the median drops to 63% (EA) and 73% (Ubisoft). The €20–40 band, where the client sits, ranges from 80 to 86%. Since the price is the *current* list price, **survivorship bias** is possible: older, cheap games that still sell are often classics.

### 11.3 Q3 · Which games keep their players?

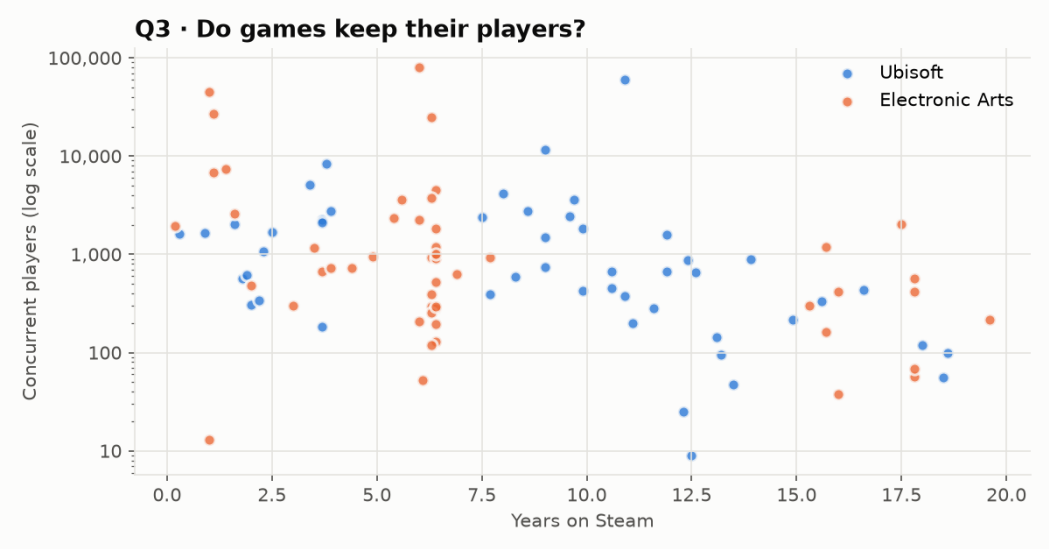



**Figure 6:** Years on Steam and concurrent players (log scale)

The four most-played games aged 5 years or more are **free-to-play**: *Apex Legends* (79,893 concurrent players), *Rainbow Six Siege* (60,193), *The Sims 4* and *Brawlhalla*. Among paid games, multiplayer, co-op (*It Takes Two*, 95% positive) and large role-playing games are the ones that last.

### 11.4 Q4 · Do French-speaking players judge differently?

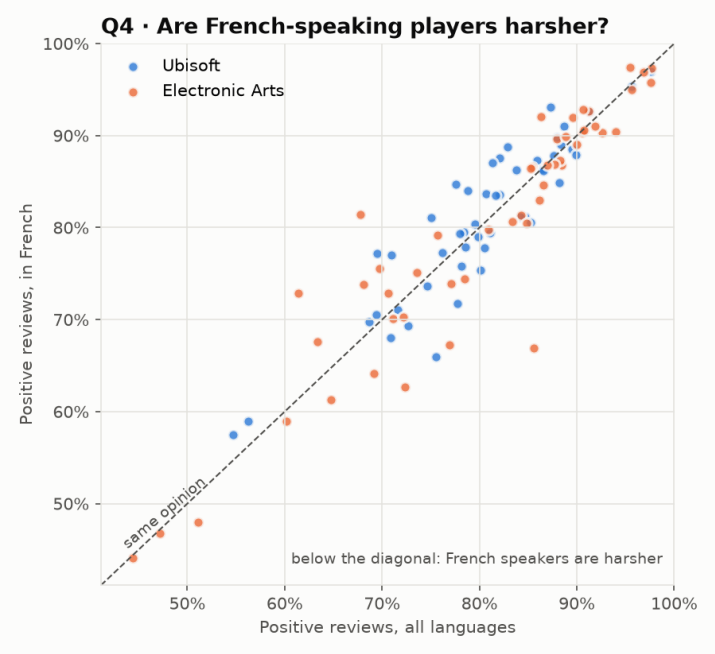



**Figure 7:** Satisfaction in French and in all languages, per game

French speakers are slightly more generous, by +2.5 points towards EA and +2.1 towards Ubisoft. They are clearly more positive on 9 Ubisoft games and harsher on only 2. Yet they write only **2% of reviews**.

### 11.5 Recommendations to the client

**For Lumen Forge Games**

1. **Price:** €25 sits in a well-rated band. Raising the price raises expectations.
2. **Launch is not a final verdict:** plan a patch roadmap. A reputation can recover.
3. **Monitor reviews month by month:** a crisis can strike years after release.
4. **Lifetime:** content updates or a co-op mode extend a game's life.
5. **Market:** localise into French, but aim international, which drives Steam reputation.

## 12 Limitations and technical debt

| Limitation | Consequence |
|---|---|
| Steam only | Findings apply to Steam players, not other platforms |
| Snapshot (Wednesday, around 7 pm) | No seasonality. Discounts not measurable. |
| Current list price | Possible survivorship bias (Q2) |
| Years on Steam | Differs from actual age for re-released games (Q1b, Q3) |
| "French speakers" = reviews in French | Country and native language are unknown |
| Correlation | Does not prove causation |
| **Technical debt accepted at level 0** | |
| No unit tests | Introduced in project 1 (`pytest`) |
| No scheduling | Manual runs. Orchestration planned for project 3. |
| Non-deterministic discovery | Reservation R1. Crawl the publisher's full catalogue. |
| Full refresh | No history between runs. Incremental loading planned for project 2. |

## 13 Lessons learned

- **Scope before coding:** a vague request became four measurable questions, each tied to a client decision.
- **Test APIs before promising:** the `day_range` parameter does not do what its name suggests.
- **Distrust perfect results:** "0 exclusions" hid a traceability gap.
- **Design for failure:** thanks to the cache, an interruption was resolved within a minute.
- **State what the data cannot tell:** the discount question remains open, and the report says so.

## 14 Roadmap

| Improvement | Benefit |
|---|---|
| Tests and PostgreSQL | Robustness and a server database (portfolio project 1) |
| Daily incremental loading | Price history, seasonality, discounts (project 2) |
| Orchestration and dbt | Scheduling, *bronze*, *silver*, *gold* layers (project 3) |
| Reputation alert | Notify when a month's share of positive reviews drops below a threshold |
| Panel of indie publishers | A comparison closer to the client's size |

## A Data dictionary: `games` table

| Column | Type | Description |
|---|---|---|
| `app_id` | INTEGER | Steam identifier of the game (primary key) |
| `name` | VARCHAR | Game title |
| `publisher_group` | VARCHAR | Publisher analysed: Ubisoft or Electronic Arts |
| `publishers` / `developers` | VARCHAR | Publisher(s) and studio(s) declared on Steam |
| `is_free` | BOOLEAN | Free-to-play game |
| `price_initial_eur` | DECIMAL | Current list price excluding discounts, in euros (not the launch price) |
| `price_final_eur` | DECIMAL | Price paid on extraction day |
| `discount_percent` | INTEGER | Current discount (%) |
| `steam_release_date` | DATE | Release date on Steam |
| `years_on_steam` | DECIMAL | Years on Steam at extraction date |
| `genres` | VARCHAR | Steam genres, in French |
| `metacritic_score` | INTEGER | Press score, when available |
| `owners_estimate_low/high` | BIGINT | SteamSpy range of estimated owners |
| `extracted_on` | DATE | Extraction date |

Full dictionary of the five tables: `docs/dictionnaire-de-donnees.md` (in French).

## B Architecture decision records (ADR)

| | |
|---|---|
| ADR 001 | Technology stack: DuckDB, pandas, `requests`, full refresh |
| ADR 002 | Extraction strategy: two passes, dated raw layer, cache, minimisation at the source |
| ADR 003 | Player count source: official Steam API rather than SteamSpy |

## C Repository structure

```
ubisoft-vs-ea-steam-etl/
|-- src/          extract.py, transform.py, load.py, report.py, pipeline.py, config.py
|-- sql/          00_schema.sql, q1a ... q4b (10 queries), controle_recette.sql
|-- docs/
|   |-- mission/      brief, scoping minutes, statement of work (French)
|   |-- decisions/    ADR 001 to 003
|   |-- journal/      dated work log
|   |-- livraison/    PDF reports, resultats.md, charts, acceptance sign-off
|   `-- rapport/      LaTeX sources of this report
|-- data/         raw batches and DuckDB database (not versioned, rebuilt by the pipeline)
|-- README.md  CHANGELOG.md  LICENSE  requirements.txt
```

## D Glossary

| | |
|---|---|
| **ETL** | Extract, Transform, Load |
| **Raw layer** | Data kept exactly as received from the sources, unchanged |
| **Idempotency** | Property of a process that gives the same result whether run once or several times |
| **Full refresh** | Complete replacement of the tables on every run |
| **Atomic write** | A write that either fully succeeds or does not happen at all |
| **ADR** | Architecture Decision Record: a short note tracing a technical decision and its rationale |
| **Acceptance** | Formal check, by the client, that the deliverable meets the agreed criteria |
| ***Review bombing*** | Sudden, massive wave of negative reviews, often in reaction to a publisher decision |